

09:00-09:50 (쉬는 시간 11:30-12:00)

# PR 자동화는 target을 정확히 읽는 것부터 시작합니다

repository·PR number·head SHA·diff를 fixture에서 읽는다.

---

진행

강의 12분 · 강사 시연 10분 · 소프트웨어  
실습 23분 · 실행 확인 5분

사용 파일

data/day4\_github/pr\_fixture.json  
src/course\_services/github\_service.py

확인할 결과

pr\_target.json

# 4일차는 여덟 개 차시로 하나의 서비스를 완성합니다

오전 3개 차시 뒤 30분 휴식, 12:00-13:00 점심, 오후 5개 차시 뒤 휴식과 Q&A로 마무리합니다.

| 시간          | 차시  | 배우는 내용                                    | 진행                                     | 확인할 결과                    |
|-------------|-----|-------------------------------------------|----------------------------------------|---------------------------|
| 09:00-09:50 | 1차시 | PR 자동화는 target을 정확히 읽는 것부터 시작합니다          | 강의 12<br>시연 10<br>소프트웨어 실습 23<br>확인 5분 | pr_target.json            |
| 09:50-10:40 | 2차시 | 토큰은 기능이 아니라 권한과 만료를 가진 비밀입니다              | 강의 12<br>시연 10<br>소프트웨어 실습 23<br>확인 5분 | .env.sample               |
| 10:40-11:30 | 3차시 | REST client는 status code를 복구 결정으로 바꿔야 합니다 | 강의 12<br>시연 10<br>소프트웨어 실습 23<br>확인 5분 | github_read_result.json   |
| 13:00-13:50 | 4차시 | LangGraph state에는 다음 결정을 위한 값만 둡니다        | 강의 12<br>시연 10<br>소프트웨어 실습 23<br>확인 5분 | compiled_review_graph     |
| 13:50-14:40 | 5차시 | 재시작 가능한 workflow는 같은 일을 두 번 하지 않습니다       | 강의 12<br>시연 10<br>소프트웨어 실습 23<br>확인 5분 | checkpoint_and_audit.json |
| 14:40-15:30 | 6차시 | Human Approval은 버튼보다 검토할 정보가 중요합니다        | 강의 12<br>시연 10<br>소프트웨어 실습 23<br>확인 5분 | review_decision.json      |
| 15:30-16:20 | 7차시 | Dry-run은 실제 게시 직전의 완성된 결과여야 합니다           | 강의 12<br>시연 10<br>소프트웨어 실습 23<br>확인 5분 | review_comment_plan.json  |
| 16:20-17:10 | 8차시 | 실제 GitHub 쓰기는 본인 sandbox에서 한 건만 실행합니다     | 강의 12<br>시연 10<br>소프트웨어 실습 23<br>확인 5분 | day4_audit_record.json    |

11:30-12:00 쉬는 시간 · 12:00-13:00 점심시간 · 17:10-17:40 쉬는 시간 · 17:40-18:00 Q&A·실행 복구

대상 commit이 바뀌면 이전 line mapping과 review comment가 stale해진다.

repository·PR number·head SHA·diff를 fixture에서 읽는다.

---

이번 차시의 확인 결과: pr\_target.json

# 이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — repository·PR number·head SHA·diff를 fixture에서 읽는다.
- 02 — 화면에 보이는 PR 제목만 믿고 repo·number·SHA를 확인하지 않는다. 왜 위험한지 설명한다.
- 03 — pr\_target.json에서 정상과 실패 상태를 직접 확인한다.

# 처음 보는 용어는 이 네 가지 역할만 구분합니다

| 용어             | 이 수업에서 쓰는 뜻      |
|----------------|------------------|
| PR             | 변경을 검토·병합하는 요청   |
| Commit         | 특정 시점의 변경 묶음     |
| Head SHA       | PR 최신 commit 식별자 |
| Review comment | diff line에 붙는 의견 |

# 입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

---

data/day4\_github/pr\_fixture.json

PROCESS

---

fixture load → target validate → head SHA 고정

OUTPUT

---

pr\_target.json

VERIFY

---

synthetic fixture가 PullRequestTarget으로 validate된다.  
필드 누락이나 workspace 밖 fixture는 구조화 오류다.

## 실행 흐름은 다섯 단계로 고정합니다

01

fixture load

02

target validate

03

head SHA 고정

04

diff load

05

read-only result

마지막 판단은 pr\_target.json에서 사람이 확인합니다.

# 코드보다 먼저 성공·실패 계약을 정합니다

|       |    |                                                    |
|-------|----|----------------------------------------------------|
| 정상    | —— | synthetic fixture가 PullRequestTarget으로 validate된다. |
| 경계    | —— | 필드 누락이나 workspace 밖 fixture는 구조화 오류다.              |
| 외부 쓰기 | —— | 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출       |
| 기록    | —— | status·error_code·입력 식별자·pr_target.json            |



# 어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

```
data/day4_github/pr_fixture.json
```

구현 코드

```
src/course_services/github_service.py
```

검증·test

```
data/day3_review_cases/unsafe_pr.diff
```

따라하기 notebook

```
materials/day4/day4_service_lab.ipynb
```

## PR 자동화는 target을 정확히 읽는 것부터 시작합니다

대상 commit이 바뀌면 이전 line mapping과 review comment가 stale해진다.

쓰기 전 target 세 필드를 payload와 승인 화면에 다시 표시한다.

이 차시에서  
버릴 오해

화면에 보이는 PR  
제목만 믿고  
repo·number·SHA를  
확인하지 않는다.

# 비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

| 구분             | 역할               | 이번 차시의 판단 기준     |
|----------------|------------------|------------------|
| PR             | 변경을 검토·병합하는 요청   | 결정론 test로 먼저 확인  |
| Commit         | 특정 시점의 변경 묶음     | adapter 경계를 확인   |
| Head SHA       | PR 최신 commit 식별자 | 비용·지연·권한을 확인     |
| Review comment | diff line에 붙는 의견 | 사람 판단과 운영 기록을 확인 |

# 가장 먼저 재현할 실패는 이것입니다

실패

화면에 보이는 PR 제목만 믿고  
repo·number·SHA를 확인하지  
않는다.

사용자 영향

대상 commit이 바뀌면 이전 line mapping과  
review comment가 stale해진다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

# 복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: 필드 누락이나 workspace 밖 fixture는 구조화 오류다.
- 02 — 중단: 화면에 보이는 PR 제목만 믿고 repo·number·SHA를 확인하지 않는다.
- 03 — 복구: 쓰기 전 target 세 필드를 payload와 승인 화면에 다시 표시한다.
- 04 — 증거: pr\_target.json과 test 결과를 함께 남긴다.

# Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

fixture loader에 필수  
field-workspace path test를  
추가한다.

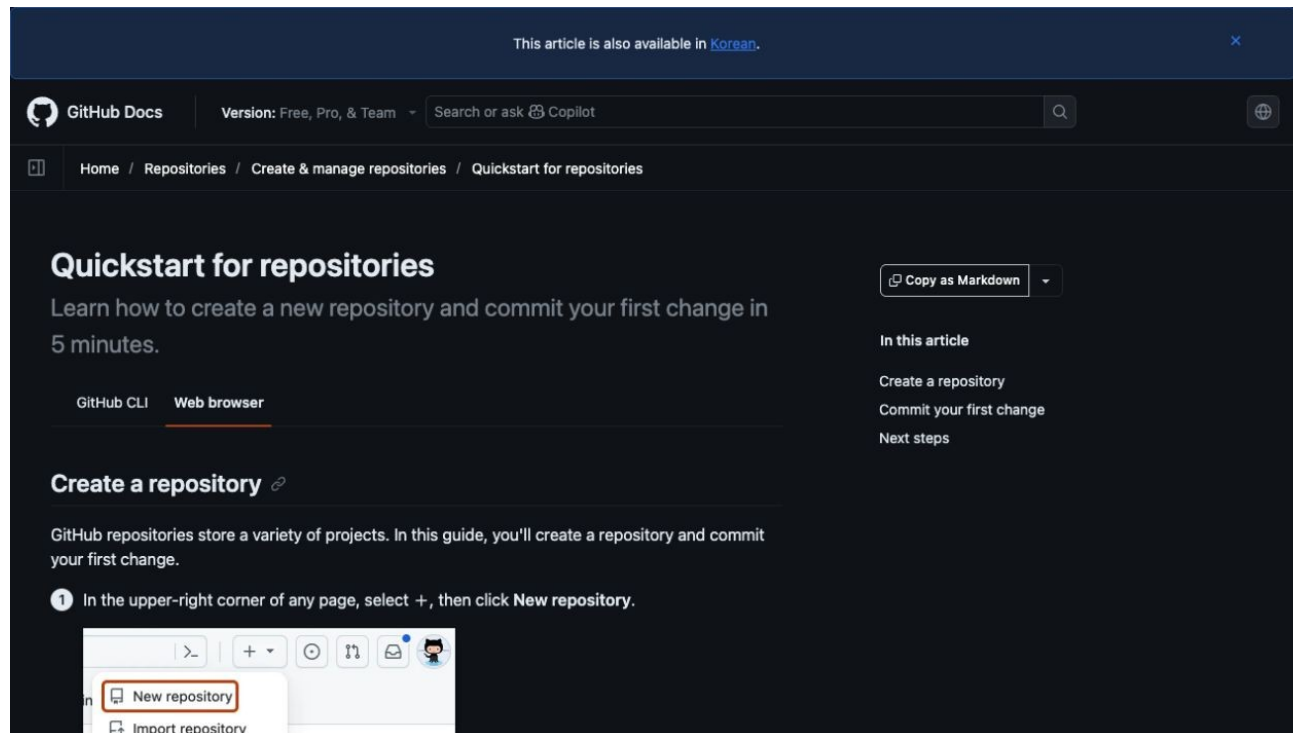
저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge



# Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

fixture loader에 필수 field·workspace path test를 추가한다.

허용 경로

src/course\_services/github\_service.py  
data/day3\_review\_cases/unsafe\_pr.diff

완료 조건

synthetic fixture가 PullRequestTarget으로 validate된다.  
필드 누락이나 workspace 밖 fixture는 구조화 오류다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

# 생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01      **Scope**                      —      허용 경로 밖 파일이 바뀌지 않았는가
- 02      **Focused test**              —      synthetic fixture가 PullRequestTarget으로 validate된다.
- 03      **Boundary test**              —      필드 누락이나 workspace 밖 fixture는 구조화 오류다.
- 04      **Human merge**              —      Codex 리뷰와 test 통과는 자동 승인이 아니다



# 강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: `data/day4_github/pr_fixture.json`
- 02 — 구현 확인: `src/course_services/github_service.py`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py -k github`
- 04 — 성공 신호: `synthetic fixture`가 `PullRequestTarget`으로 `validate`된다.

## 같은 저장소에서 이 명령을 실행합니다

```
$ python -m pytest -q tests/test_course_services.py -k github
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

# 완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED\_FAILURE

ARTIFACT

pr\_target.json

사람이 확인할 세 줄

1. synthetic fixture가 PullRequestTarget으로 validate된다.
2. 필드 누락이나 workspace 밖 fixture는 구조화 오류다.
3. external\_write=false

# 강사는 실패 한 건도 바로 재현하고 복구합니다

재현

화면에 보이는 PR 제목만 믿고  
repo-number-SHA를 확인하지 않는다.

복구

쓰기 전 target 세 필드를 payload와  
승인 화면에 다시 표시한다.

확인: 필드 누락이나 workspace 밖 fixture는 구조화 오류다.

# 이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

materials/day4/day4\_service\_lab.ipynb

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

synthetic fixture가 PullRequestTarget으로 validate된다.  
pr\_target.json 저장

## STEP 1 · 입력과 설정을 확인합니다

파일: data/day4\_github/pr\_fixture.json  
변수: 실행 경로·provider·dry-run 여부

---

결과 파일: pr\_target.json

## STEP 2 · 정상 경로를 실행합니다

```
python -m pytest -q tests/test_course_services.py -k github
```

성공 신호: synthetic fixture가 PullRequestTarget으로 validate된다.

---

결과 파일: pr\_target.json

## STEP 3 · 경계값을 바꿔 실패를 확인합니다

필드 누락이나 workspace 밖 fixture는 구조화 오류다.  
복구: 쓰기 전 target 세 필드를 payload와 승인 화면에 다시 표시한다.

---

결과에 `error_code`와 `external_write=false`가 보이면 완료



## 정상 case를 focused test로 확인합니다

NORMAL

synthetic fixture가 PullRequestTarget으로 validate된다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

## 가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

필드 누락이나 workspace 밖 fixture는 구조화 오류다.

- 01 — raw traceback 대신 stable error\_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

# 재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

사내 repo PR을 읽어 검토 대기열을 만드는 read-only bot

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — pr\_target.json을 운영 검토 자료로 사용

# 구직자는 제품 판단과 검증 증거를 함께 보여줍니다

## 사내 repo PR을 읽어 검토 대기열을 만드는 read-only bot

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — pr\_target.json과 README 재현 절차를 제출

# 서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

## 품질

---

무엇을 synthetic fixture가 PullRequestTarget으로 validate된다.로 정의하는가

## 안전

---

어디에서 사람 승인과 external\_write=false를 확인하는가

## 복구

---

쓰기 전 target 세 필드를 payload와 승인 화면에 다시 표시한다.

## 관측

---

pr\_target.json에서 어떤 status\_error\_code를 보는가

# 1차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: PR 자동화는 target을 정확히 읽는 것부터 시작합니다이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py -k github`
- 03 — 증거: `pr_target.json`과 정상·실패 test 결과가 남아 있다.

다음 차시: 토큰은 기능이 아니라 권한과 만료를 가진 비밀입니다

# 토큰은 기능이 아니라 권한과 만료일을 가진 비밀입니다

fine-grained token과 최소 repository permission을 설명한다.

---

## 진행

강의 12분 · 강사 시연 10분 · 소프트웨어  
실습 23분 · 실행 확인 5분

## 사용 파일

.env.sample  
.gitignore

## 확인할 결과

.env.sample

## 2차시는 앞의 결과를 이어받아 다음 상태를 만듭니다

|       |      |                                           |
|-------|------|-------------------------------------------|
| 입력    | ———— | pr_target.json                            |
| 이번 차시 | ———— | 토큰은 기능이 아니라 권한과 만료를 가진 비밀입니다              |
| 결과    | ———— | .env.sample                               |
| 다음 연결 | ———— | REST client는 status code를 복구 결정으로 바꿔야 합니다 |



교육용 token 하나가 실수로 commit되면 저장소 전체 위협으로 번진다.

fine-grained token과 최소 repository permission을 설명한다.

---

이번 차시의 확인 결과: .env.sample

# 이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — fine-grained token과 최소 repository permission을 설명한다.
- 02 — token 값을 notebook output이나 screenshot에 노출한다. 왜 위험한지 설명한다.
- 03 — .env.sample에서 정상과 실패 상태를 직접 확인한다.

## 처음 보는 용어는 이 네 가지 역할만 구분합니다

| 용어              | 이 수업에서 쓰는 뜻       |
|-----------------|-------------------|
| PAT             | 개인 API 접근 토큰      |
| Fine-grained    | 대상과 권한을 좁힌 token  |
| Secret          | 소스·로그에 남기면 안 되는 값 |
| Least privilege | 필요한 최소 권한만 부여     |

# 입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

---

.env.sample

PROCESS

---

권한 목록 → sandbox repo → .env local

OUTPUT

---

.env.sample

VERIFY

---

.env는 ignore되고 .env.sample만 추적된다.  
token이 없으면 401을 재시도하지 않고 실행을  
중단한다.

## 실행 흐름은 다섯 단계로 고정합니다

01

권한 목록

02

sandbox repo

03

.env local

04

.gitignore

05

완료·폐기

마지막 판단은 .env.sample에서 사람이 확인합니다.

# 코드보다 먼저 성공·실패 계약을 정합니다

- 정상 ——— .env는 ignore되고 .env.sample만 추적된다.
- 경계 ——— token이 없으면 401을 재시도하지 않고 실행을 중단한다.
- 외부 쓰기 ——— 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
- 기록 ——— status·error\_code·입력 식별자·env.sample

# 어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

`.env.sample`

구현 코드

`.gitignore`

검증·test

`AGENTS.md`

따라하기 notebook

`data/day4_github/pr_fixture.json`

# 토큰은 기능이 아니라 권한과 만료를 가진 비밀입니다

교육용 token 하나가 실수로 commit되면 저장소 전체 위협으로 번진다.

존재 여부만 bool로 확인하고 노출된 token은 즉시 폐기·재발급한다.

이 차시에서  
버릴 오해

token 값을 notebook  
output이나 screenshot  
에 노출한다.



## 비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

| 구분              | 역할                | 이번 차시의 판단 기준     |
|-----------------|-------------------|------------------|
| PAT             | 개인 API 접근 토큰      | 결정론 test로 먼저 확인  |
| Fine-grained    | 대상과 권한을 좁힌 token  | adapter 경계를 확인   |
| Secret          | 소스·로그에 남기면 안 되는 값 | 비용·지연·권한을 확인     |
| Least privilege | 필요한 최소 권한만 부여     | 사람 판단과 운영 기록을 확인 |

# 가장 먼저 재현할 실패는 이것입니다

실패

token 값을 notebook output이나  
screenshot에 노출한다.

사용자 영향

교육용 token 하나가 실수로 commit되면  
저장소 전체 위협으로 번진다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

## 복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: token이 없으면 401을 재시도하지 않고 실행을 중단한다.
- 02 — 중단: token 값을 notebook output이나 screenshot에 노출한다.
- 03 — 복구: 존재 여부만 bool로 확인하고 노출된 token은 즉시 폐기·재발급한다.
- 04 — 증거: .env.sample과 test 결과를 함께 남긴다.

# Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

secret을 읽거나 출력하지 않는  
범위에서 config validation만  
구현한다.

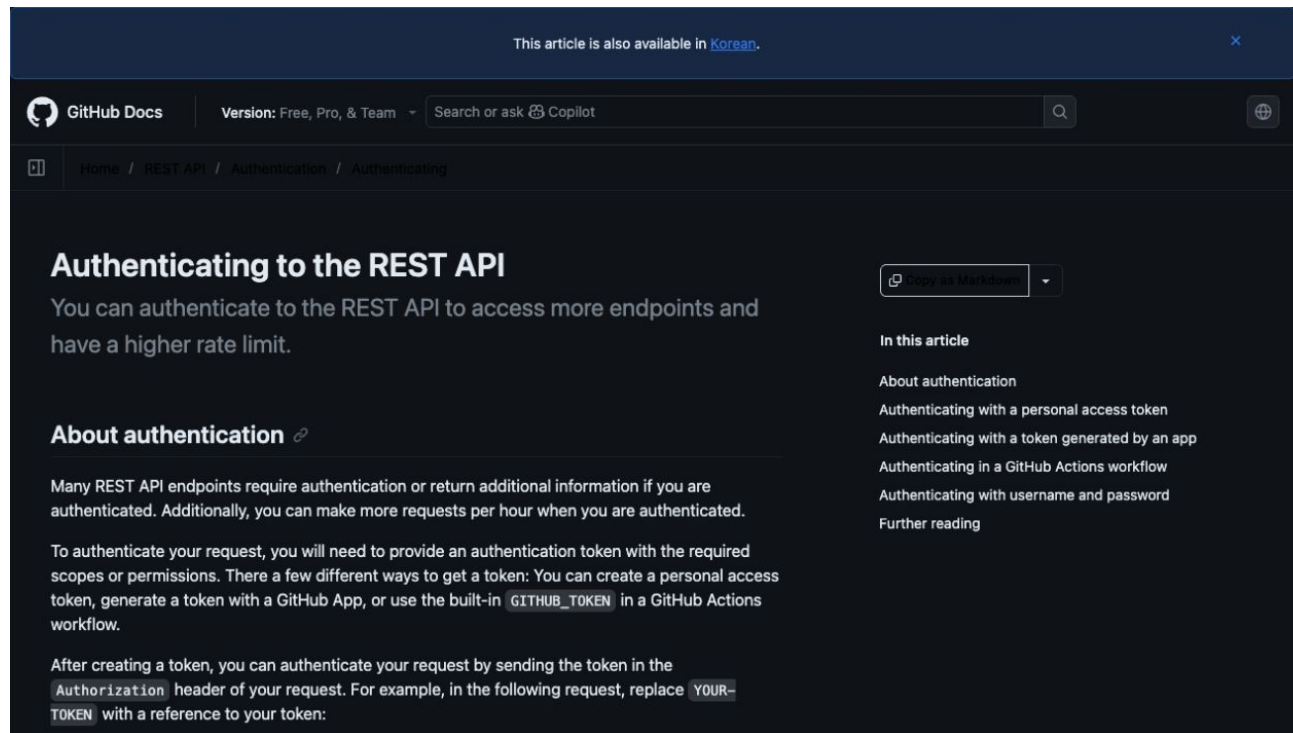
저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge



# Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

secret을 읽거나 출력하지 않는 범위에서 config validation만 구현한다.

허용 경로

.gitignore  
AGENTS.md

완료 조건

.env는 ignore되고 .env.sample만 추적된다.  
token이 없으면 401을 재시도하지 않고 실행을 중단한다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

# 생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01      **Scope**                      —      허용 경로 밖 파일이 바뀌지 않았는가
- 02      **Focused test**              —      .env는 ignore되고 .env.sample만 추적된다.
- 03      **Boundary test**              —      token이 없으면 401을 재시도하지 않고 실행을 중단한다.
- 04      **Human merge**                —      Codex 리뷰와 test 통과는 자동 승인이 아니다

# 강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: `.env.sample`
- 02 — 구현 확인: `.gitignore`
- 03 — 실행: `git check-ignore -v .env`
- 04 — 성공 신호: `.env`는 ignore되고 `.env.sample`만 추적된다.

## 같은 저장소에서 이 명령을 실행합니다

```
$ git check-ignore -v .env
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지



# 완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED\_FAILURE

ARTIFACT

.env.sample

사람이 확인할 세 줄

1. .env는 ignore되고 .env.sample만 추적된다.
2. token이 없으면 401을 재시도하지 않고 실행을 중단한다.
3. external\_write=false

## 강사는 실패 한 건도 바로 재현하고 복구합니다

재현

token 값을 notebook output이나  
screenshot에 노출한다.

복구

존재 여부만 bool로 확인하고 노출된  
token은 즉시 폐기·재발급한다.

확인: token이 없으면 401을 재시도하지 않고 실행을 중단한다.

# 이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

data/day4\_github/pr\_fixture.json

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

.env는 ignore되고 .env.sample만 추적된다.  
.env.sample 저장

# STEP 1 · 입력과 설정을 확인합니다

파일: .env.sample

변수: 실행 경로·provider·dry-run 여부

---

결과 파일: .env.sample

## STEP 2 · 정상 경로를 실행합니다

```
git check-ignore -v .env
```

성공 신호: .env는 ignore되고 .env.sample만 추적된다.

---

결과 파일: .env.sample

## STEP 3 · 경계값을 바꿔 실패를 확인합니다

token이 없으면 401을 재시도하지 않고 실행을 중단한다.  
복구: 존재 여부만 bool로 확인하고 노출된 token은 즉시 폐기·재발급한다.

---

결과에 error\_code와 external\_write=false가 보이면 완료

## 정상 case를 focused test로 확인합니다

NORMAL

.env는 ignore되고 .env.sample만 추적된다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

## 가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

token이 없으면 401을 재시도하지 않고 실행을 중단한다.

- 01 — raw traceback 대신 stable error\_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가



# 재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

교육에서는 mock, 운영에서는 App token·short-lived credential 고려

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — .env.sample을 운영 검토 자료로 사용

# 구직자는 제품 판단과 검증 증거를 함께 보여줍니다

교육에서는 mock, 운영에서는 App token·short-lived credential 고려

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — .env.sample과 README 재현 절차를 제출

# 서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

## 품질

---

무엇을 .env는 ignore되고 .env.sample만 추적된다.  
로 정의하는가

## 안전

---

어디에서 사람 승인과 external\_write=false를  
확인하는가

## 복구

---

존재 여부만 bool로 확인하고 노출된 token은 즉시  
폐기·재발급한다.

## 관측

---

.env.sample에서 어떤 status\_error\_code를 보는가

## 2차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: 토큰은 기능이 아니라 권한과 만료를 가진 비밀입니다. 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `git check-ignore -v .env`
- 03 — 증거: `.env.sample`과 정상·실패 test 결과가 남아 있다.

다음 차시: REST client는 status code를 복구 결정으로 바꿔야 합니다

# REST client는 status code를 복구 결정으로 바꿔야 합니다

read-only PR 조회와 pagination·rate limit 계약을 만든다.

---

## 진행

강의 12분 · 강사 시연 10분 · 소프트웨어  
실습 23분 · 실행 확인 5분

## 사용 파일

src/course\_services/github\_service.py  
materials/day4/day4\_service\_lab.ipynb

## 확인할 결과

github\_read\_result.json

## 3차시는 앞의 결과를 이어받아 다음 상태를 만듭니다

|       |     |                                           |
|-------|-----|-------------------------------------------|
| 입력    | ——— | .env.sample                               |
| 이번 차시 | ——— | REST client는 status code를 복구 결정으로 바꿔야 합니다 |
| 결과    | ——— | github_read_result.json                   |
| 다음 연결 | ——— | LangGraph state에는 다음 결정을 위한 값만 둡니다        |

모든 오류를 같은 retry로 처리하면 인증 문제와 rate limit이 장애를 키운다.

read-only PR 조회와 pagination·rate limit 계약을 만든다.

---

이번 차시의 확인 결과: github\_read\_result.json

# 이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — read-only PR 조회와 pagination·rate limit 계약을 만든다.
- 02 — 401·403·404·422·429를 모두 세 번 retry한다. 왜 위험한지 설명한다.
- 03 — github\_read\_result.json에서 정상과 실패 상태를 직접 확인한다.



## 처음 보는 용어는 이 네 가지 역할만 구분합니다

| 용어          | 이 수업에서 쓰는 뜻             |
|-------------|-------------------------|
| REST        | HTTP resource 단위 API 방식 |
| Pagination  | 여러 페이지로 결과를 나눔          |
| Rate limit  | 시간당 요청 제한               |
| Retry-After | 다시 요청 가능한 시각 힌트         |

# 입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

---

src/course\_services/github\_service.py

OUTPUT

---

github\_read\_result.json

PROCESS

---

GET request → status classify → pagination

VERIFY

---

200 응답은 target·diff·rate metadata를 반환한다.  
429만 Retry-After와 최대 횟수 안에서 재시도한다.

## 실행 흐름은 다섯 단계로 고정합니다

01

GET request

02

status classify

03

pagination

04

bounded backoff

05

structured result

마지막 판단은 `github_read_result.json`에서 사람이 확인합니다.

# 코드보다 먼저 성공·실패 계약을 정합니다

|       |    |                                                  |
|-------|----|--------------------------------------------------|
| 정상    | —— | 200 응답은 target·diff·rate metadata를 반환한다.         |
| 경계    | —— | 429만 Retry-After와 최대 횟수 안에서 재시도한다.               |
| 외부 쓰기 | —— | 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출     |
| 기록    | —— | status·error_code·입력 식별자·github_read_result.json |

# 어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

```
src/course_services/github_service.py
```

구현 코드

```
materials/day4/day4_service_lab.ipynb
```

검증·test

```
AGENTS.md
```

따라하기 notebook

```
output/day4-github/read-result.json
```

## REST client는 status code를 복구 결정으로 바꿔야 합니다

모든 오류를 같은 retry로 처리하면 인증 문제와 rate limit  
이 장애를 키운다.

재인증·권한 확인·target 확인·diff 갱신·대기 경로를 error code별로 나눈다.

이 차시에서  
버릴 오해

401·403·404·422·429  
를 모두 세 번 retry  
한다.

## 비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

| 구분          | 역할                      | 이번 차시의 판단 기준     |
|-------------|-------------------------|------------------|
| REST        | HTTP resource 단위 API 방식 | 결정론 test로 먼저 확인  |
| Pagination  | 여러 페이지로 결과를 나눔          | adapter 경계를 확인   |
| Rate limit  | 시간당 요청 제한               | 비용·지연·권한을 확인     |
| Retry-After | 다시 요청 가능한 시각 힌트         | 사람 판단과 운영 기록을 확인 |

# 가장 먼저 재현할 실패는 이것입니다

실패

401·403·404·422·429를 모두 세 번 retry한다.

사용자 영향

모든 오류를 같은 retry로 처리하면 인증 문제와 rate limit이 장애를 키운다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.



## 복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: 429만 Retry-After와 최대 횟수 안에서 재시도한다.
- 02 — 중단: 401·403·404·422·429를 모두 세 번 retry한다.
- 03 — 복구: 재인증·권한 확인·target 확인·diff 갱신·대기 경로를 error code별로 나눈다.
- 04 — 증거: github\_read\_result.json과 test 결과를 함께 남긴다.

# Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

fake HTTP response로 각 status code의 결정론 test를 작성한다.

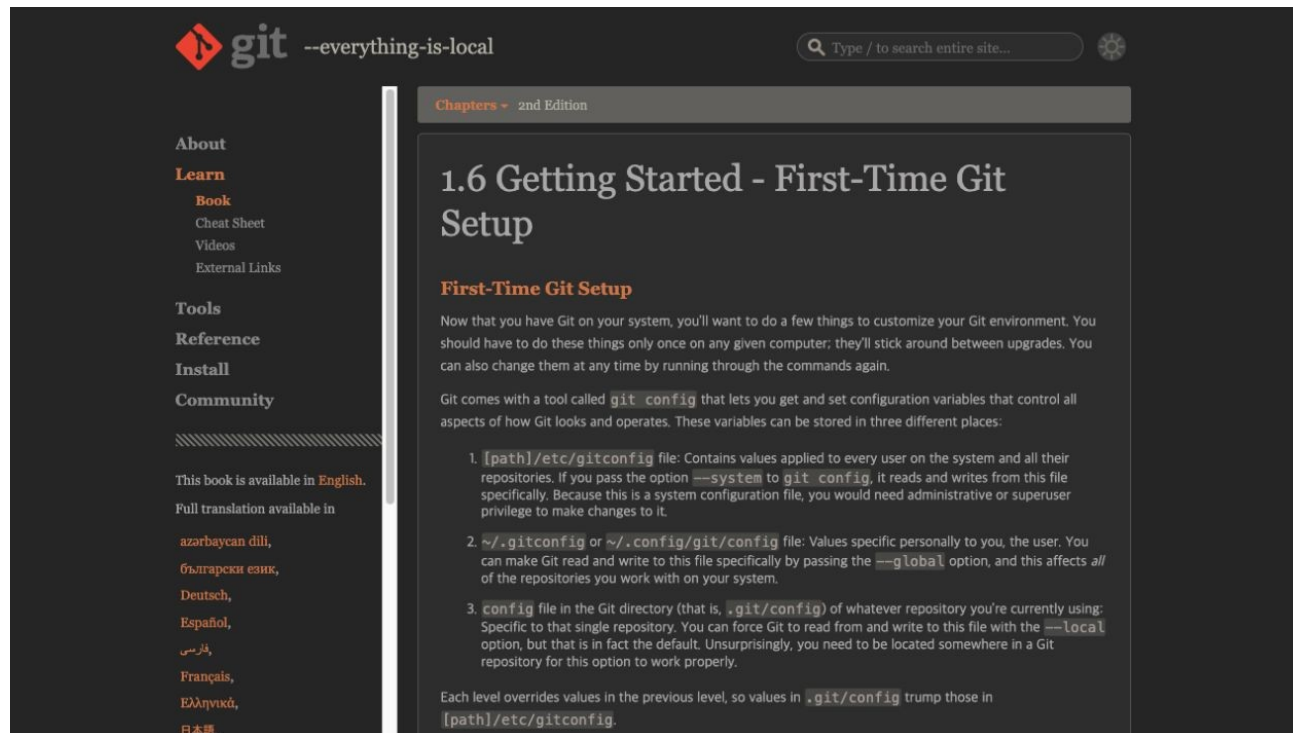
저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge



# Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

fake HTTP response로 각 status code의 결정론 test를 작성한다.

허용 경로

materials/day4/day4\_service\_lab.ipynb  
AGENTS.md

완료 조건

200 응답은 target-diff-rate metadata를 반환한다.  
429만 Retry-After와 최대 횟수 안에서 재시도한다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

# 생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01      **Scope**                      —      허용 경로 밖 파일이 바뀌지 않았는가
- 02      **Focused test**                —      200 응답은 target·diff·rate metadata를 반환한다.
- 03      **Boundary test**                —      429만 Retry-After와 최대 횟수 안에서 재시도한다.
- 04      **Human merge**                —      Codex 리뷰와 test 통과는 자동 승인이 아니다

# 강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: `src/course_services/github_service.py`
- 02 — 구현 확인: `materials/day4/day4_service_lab.ipynb`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py`
- 04 — 성공 신호: 200 응답은 `target·diff·rate metadata`를 반환한다.

## 같은 저장소에서 이 명령을 실행합니다

```
$ python -m pytest -q tests/test_course_services.py
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

# 완료 여부는 화면이 아니라 결과 계약으로 확인합니다

## STATUS

SUCCESS 또는 EXPECTED\_FAILURE

## ARTIFACT

github\_read\_result.json

사람이 확인할 세 줄

1. 200 응답은 target·diff·rate metadata를 반환한다.
2. 429만 Retry-After와 최대 횟수 안에서 재시도한다.
3. external\_write=false

# 강사는 실패 한 건도 바로 재현하고 복구합니다

재현

401·403·404·422·429를 모두 세 번  
retry한다.

복구

재인증·권한 확인·target 확인·diff 갱신·  
대기 경로를 error code별로 나눈다.

확인: 429만 Retry-After와 최대 횟수 안에서 재시도한다.



# 이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

output/day4-github/read-result.json

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

200 응답은 target·diff·rate metadata를 반환한다.  
github\_read\_result.json 저장

## STEP 1 · 입력과 설정을 확인합니다

파일 : `src/course_services/github_service.py`  
변수 : 실행 경로·provider·dry-run 여부

---

결과 파일 : `github_read_result.json`

## STEP 2 · 정상 경로를 실행합니다

```
python -m pytest -q tests/test_course_services.py
```

성공 신호: 200 응답은 target-diff-rate metadata를 반환한다.

---

결과 파일: github\_read\_result.json

## STEP 3 · 경계값을 바꿔 실패를 확인합니다

429만 Retry-After와 최대 횟수 안에서 재시도한다.

복구: 재인증·권한 확인·target 확인·diff 갱신·대기 경로를 error code별로 나눈다.

---

결과에 `error_code`와 `external_write=false`가 보이면 완료

## 정상 case를 focused test로 확인합니다

NORMAL

200 응답은 target·diff·rate metadata를 반환한다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

## 가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

429만 Retry-After와 최대 횟수 안에서 재시도한다.

- 01 — raw traceback 대신 stable error\_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

# 재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

## PR 정보를 안전하게 수집하는 ingestion adapter

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — github\_read\_result.json을 운영 검토 자료로 사용

# 구직자는 제품 판단과 검증 증거를 함께 보여줍니다

## PR 정보를 안전하게 수집하는 ingestion adapter

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — github\_read\_result.json과 README 재현 절차를 제출



# 서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

## 품질

---

무엇을 200 응답은 target·diff·rate metadata를 반환한다.로 정의하는가

## 복구

---

재인증·권한 확인·target 확인·diff 갱신·대기 경로를 error code별로 나눈다.

## 안전

---

어디에서 사람 승인과 external\_write=false를 확인하는가

## 관측

---

github\_read\_result.json에서 어떤 status·error\_code를 보는가

## 3차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: REST client는 status code를 복구 결정으로 바꿔야 합니다이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py`
- 03 — 증거: `github_read_result.json`과 정상·실패 test 결과가 남아 있다.

다음 차시: LangGraph state에는 다음 결정을 위한 값만 둡니다

# LangGraph state에는 다음 결정을 위한 값만 듭니다

PR review의 node·edge·conditional edge를 조립한다.

---

진행

강의 12분 · 강사 시연 10분 · 소프트웨어  
실습 23분 · 실행 확인 5분

사용 파일

src/langgraph\_lab.py  
src/meeting\_agent\_workflow.py

확인할 결과

compiled\_review\_graph

## 4차시는 앞의 결과를 이어받아 다음 상태를 만듭니다

|       |    |                                     |
|-------|----|-------------------------------------|
| 입력    | —— | github_read_result.json             |
| 이번 차시 | —— | LangGraph state에는 다음 결정을 위한 값만 둡니다  |
| 결과    | —— | compiled_review_graph               |
| 다음 연결 | —— | 재시작 가능한 workflow는 같은 일을 두 번 하지 않습니다 |

모든 원문과 client 객체를 state에 넣으면 checkpoint와 재시작이 불안정해진다.

PR review의 node·edge·conditional edge를 조립한다.

---

이번 차시의 확인 결과: `compiled_review_graph`

# 이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — PR review의 node·edge·conditional edge를 조립한다.
- 02 — state에 token·client·raw audio를 넣어 checkpoint에 남긴다. 왜 위험한지 설명한다.
- 03 — compiled\_review\_graph에서 정상과 실패 상태를 직접 확인한다.

## 처음 보는 용어는 이 네 가지 역할만 구분합니다

| 용어               | 이 수업에서 쓰는 뜻         |
|------------------|---------------------|
| State            | node 사이에 전달할 명시 데이터 |
| Node             | 한 책임을 실행하는 함수       |
| Edge             | 다음 실행 순서            |
| Conditional edge | 상태에 따른 분기           |

# 입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

---

src/langgraph\_lab.py

OUTPUT

---

compiled\_review\_graph

PROCESS

---

validate target → load diff → review

VERIFY

---

approve·edit·reject가 서로 다른 terminal state로 간다.  
validation error는 publish node에 도달하지 않는다.



## 실행 흐름은 다섯 단계로 고정합니다

01

validate target

02

load diff

03

review

04

human review

05

dry-run

마지막 판단은 `compiled_review_graph`에서 사람이 확인합니다.

# 코드보다 먼저 성공·실패 계약을 정합니다

|       |                                                |
|-------|------------------------------------------------|
| 정상    | approve·edit·reject가 서로 다른 terminal state로 간다. |
| 경계    | validation error는 publish node에 도달하지 않는다.      |
| 외부 쓰기 | 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출   |
| 기록    | status·error_code·입력 식별자·compiled_review_graph |

# 어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

```
src/langgraph_lab.py
```

구현 코드

```
src/meeting_agent_workflow.py
```

검증·test

```
src/course_services/github_service.py
```

따라하기 notebook

```
materials/day4/day4_service_lab.ipynb
```

## LangGraph state에는 다음 결정을 위한 값만 둡니다

모든 원문과 client 객체를 state에 넣으면 checkpoint와 재시작이 불안정해진다.

ID·status·검증 결과만 state에 두고 외부 자원은 adapter 경계에서 주입한다.

이 차시에서  
버릴 오해

state에 token·client·raw  
audio를 넣어 checkpoint  
에 남긴다.

## 비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

| 구분               | 역할                  | 이번 차시의 판단 기준     |
|------------------|---------------------|------------------|
| State            | node 사이에 전달할 명시 데이터 | 결정론 test로 먼저 확인  |
| Node             | 한 책임을 실행하는 함수       | adapter 경계를 확인   |
| Edge             | 다음 실행 순서            | 비용·지연·권한을 확인     |
| Conditional edge | 상태에 따른 분기           | 사람 판단과 운영 기록을 확인 |

# 가장 먼저 재현할 실패는 이것입니다

실패

state에 token·client·raw audio를  
넣어 checkpoint에 남긴다.

사용자 영향

모든 원문과 client 객체를 state에 넣으면  
checkpoint와 재시작이 불안정해진다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

## 복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: validation error는 publish node에 도달하지 않는다.
- 02 — 중단: state에 token·client·raw audio를 넣어 checkpoint에 남긴다.
- 03 — 복구: ID·status·검증 결과만 state에 두고 외부 자원은 adapter 경계에서 주입한다.
- 04 — 증거: compiled\_review\_graph과 test 결과를 함께 남긴다.

# Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

기존 service 함수를 node로  
감싸되 contract를 바꾸지 않게  
한다.

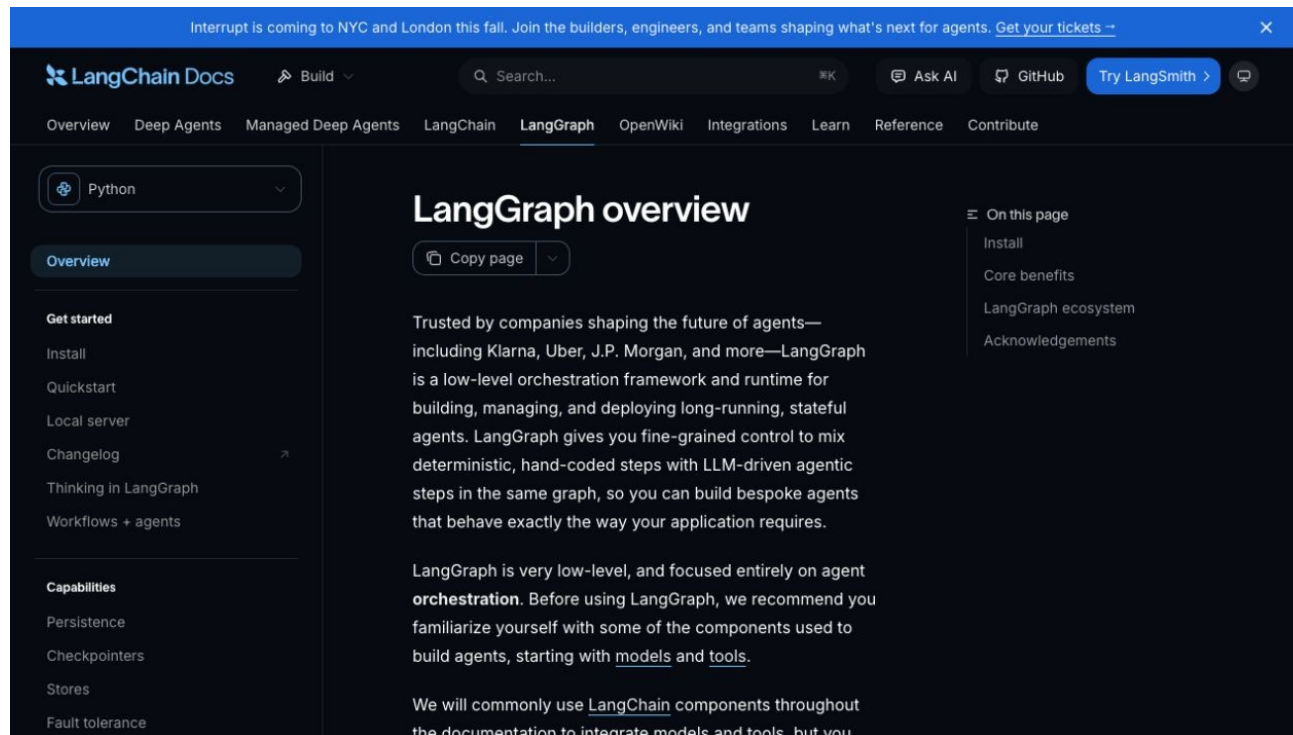
저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge





# Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

기존 service 함수를 node로 감싸되 contract를 바꾸지 않게 한다.

허용 경로

src/meeting\_agent\_workflow.py  
src/course\_services/github\_service.py

완료 조건

approve·edit·reject가 서로 다른 terminal state로 간다.  
validation error는 publish node에 도달하지 않는다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

# 생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01      **Scope**                      —      허용 경로 밖 파일이 바뀌지 않았는가
- 02      **Focused test**              —      approve·edit·reject가 서로 다른 terminal state로 간다.
- 03      **Boundary test**              —      validation error는 publish node에 도달하지 않는다.
- 04      **Human merge**              —      Codex 리뷰와 test 통과는 자동 승인이 아니다

# 강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: `src/langgraph_lab.py`
- 02 — 구현 확인: `src/meeting_agent_workflow.py`
- 03 — 실행: `python -m src.langgraph_lab --decision all`
- 04 — 성공 신호: `approve·edit·reject`가 서로 다른 terminal state로 간다.

## 같은 저장소에서 이 명령을 실행합니다

```
$ python -m src.langgraph_lab --decision all
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

# 완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED\_FAILURE

ARTIFACT

compiled\_review\_graph

사람이 확인할 세 줄

1. approve·edit·reject가 서로 다른 terminal state로 간다.
2. validation error는 publish node에 도달하지 않는다.
3. external\_write=false

## 강사는 실패 한 건도 바로 재현하고 복구합니다

재현

state에 token·client·raw audio를 넣어  
checkpoint에 남긴다.

복구

ID·status·검증 결과만 state에 두고  
외부 자원은 adapter 경계에서  
주입한다.

확인: validation error는 publish node에 도달하지 않는다.

# 이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

materials/day4/day4\_service\_lab.ipynb

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

approve·edit·reject가 서로 다른 terminal state로 간다.  
compiled\_review\_graph 저장

## STEP 1 · 입력과 설정을 확인합니다

파일: `src/langgraph_lab.py`  
변수: 실행 경로·provider·dry-run 여부

---

결과 파일: `compiled_review_graph`



## STEP 2 · 정상 경로를 실행합니다

```
python -m src.langgraph_lab --decision all
```

성공 신호: approve·edit·reject가 서로 다른 terminal state로 간다.

---

결과 파일: compiled\_review\_graph

## STEP 3 · 경계값을 바꿔 실패를 확인합니다

validation error는 publish node에 도달하지 않는다.  
복구: ID·status·검증 결과만 state에 두고 외부 자원은 adapter 경계에서  
주입한다.

---

결과에 error\_code와 external\_write=false가 보이면 완료

## 정상 case를 focused test로 확인합니다

NORMAL

approve·edit·reject가 서로 다른 terminal state로 간다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

## 가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

validation error는 publish node에 도달하지 않는다.

- 01 — raw traceback 대신 stable error\_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

# 재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

## 회의 Agent와 PR Agent가 같은 승인 패턴을 재사용

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — compiled\_review\_graph을 운영 검토 자료로 사용

# 구직자는 제품 판단과 검증 증거를 함께 보여줍니다

## 회의 Agent와 PR Agent가 같은 승인 패턴을 재사용

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — compiled\_review\_graph과 README 재현 절차를 제출

# 서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

## 품질

---

무엇을 approve·edit·reject가 서로 다른 terminal state로 간다.로 정의하는가

## 안전

---

어디에서 사람 승인과 external\_write=false를 확인하는가

## 복구

---

ID·status·검증 결과만 state에 두고 외부 자원은 adapter 경계에서 주입한다.

## 관측

---

compiled\_review\_graph에서 어떤 status·error\_code를 보는가

## 4차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: LangGraph state에는 다음 결정을 위한 값만 둡니다이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m src.langgraph_lab --decision all`
- 03 — 증거: `compiled_review_graph`과 정상·실패 test 결과가 남아 있다.

다음 차시: 재시작 가능한 workflow는 같은 일을 두 번 하지 않습니다



13:50-14:40 (쉬는 시간 17:10-17:40)

# 재시작 가능한 workflow는 같은 일을 두 번 하지 않습니다

retry·checkpoint·request ID·idempotency를 연결한다.

---

## 진행

강의 12분 · 강사 시연 10분 · 소프트웨어  
실습 23분 · 실행 확인 5분

## 사용 파일

src/course\_services/github\_service.py  
src/langgraph\_lab.py

## 확인할 결과

checkpoint\_and\_audit.json

## 5차시는 앞의 결과를 이어받아 다음 상태를 만듭니다

|       |    |                                     |
|-------|----|-------------------------------------|
| 입력    | —— | compiled_review_graph               |
| 이번 차시 | —— | 재시작 가능한 workflow는 같은 일을 두 번 하지 않습니다 |
| 결과    | —— | checkpoint_and_audit.json           |
| 다음 연결 | —— | Human Approval은 버튼보다 검토할 정보가 중요합니다  |

interrupt 전후 node가 다시 실행돼도 외부 쓰기는 한 번이어야 한다.

retry-checkpoint-request ID-idempotency를 연결한다.

---

이번 차시의 확인 결과: checkpoint\_and\_audit.json

# 이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — `retry·checkpoint·request ID·idempotency`를 연결한다.
- 02 — `timeout` 뒤 동일 `comment`를 다시 게시한다. 왜 위험한지 설명한다.
- 03 — `checkpoint_and_audit.json`에서 정상과 실패 상태를 직접 확인한다.

## 처음 보는 용어는 이 네 가지 역할만 구분합니다

| 용어          | 이 수업에서 쓰는 뜻             |
|-------------|-------------------------|
| Checkpoint  | 중단 시 state 저장           |
| Thread ID   | 같은 실행을 이어 가는 식별자        |
| Idempotency | 같은 요청을 반복해도 결과가 한 번인 성질 |
| Backoff     | 재시도 간격을 점차 늘림           |

# 입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

---

src/course\_services/github\_service.py

PROCESS

---

request ID → checkpoint → bounded retry

OUTPUT

---

checkpoint\_and\_audit.json

VERIFY

---

첫 실행은 reused=false, 두 번째는 true다.  
body나 head SHA가 달라지면 새 승인 대상으로 본다.

## 실행 흐름은 다섯 단계로 고정합니다

01

request ID

02

checkpoint

03

bounded retry

04

idempotency  
lookup

05

reuse result

마지막 판단은 `checkpoint_and_audit.json`에서 사람이 확인합니다.

# 코드보다 먼저 성공·실패 계약을 정합니다

|       |                                                       |
|-------|-------------------------------------------------------|
| 정상    | —— 첫 실행은 reused=false, 두 번째는 true다.                   |
| 경계    | —— body나 head SHA가 달라지면 새 승인 대상으로 본다.                 |
| 외부 쓰기 | —— 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출       |
| 기록    | —— status·error_code·입력 식별자·checkpoint_and_audit.json |



# 어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

```
src/course_services/github_service.py
```

구현 코드

```
src/langgraph_lab.py
```

검증·test

```
tests/test_course_services.py
```

따라하기 notebook

```
materials/day4/day4_service_lab.ipynb
```

## 재시작 가능한 workflow는 같은 일을 두 번 하지 않습니다

interrupt 전후 node가 다시 실행되도 외부 쓰기는 한 번이어야 한다.

target:SHA:body hash를 idempotency key로 저장하고 기존 remote result를 재사용한다.

이 차시에서  
버릴 오해

timeout 뒤 동일  
comment를 다시  
게시한다.

## 비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

| 구분          | 역할                      | 이번 차시의 판단 기준     |
|-------------|-------------------------|------------------|
| Checkpoint  | 중단 시 state 저장           | 결정론 test로 먼저 확인  |
| Thread ID   | 같은 실행을 이어 가는 식별자        | adapter 경계를 확인   |
| Idempotency | 같은 요청을 반복해도 결과가 한 번인 성질 | 비용·지연·권한을 확인     |
| Backoff     | 재시도 간격을 점차 늘림           | 사람 판단과 운영 기록을 확인 |

## 가장 먼저 재현할 실패는 이것입니다

실패

timeout 뒤 동일 comment를 다시  
게시한다.

사용자 영향

interrupt 전후 node가 다시 실행돼도 외부  
쓰기는 한 번이어야 한다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

## 복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: body나 head SHA가 달라지면 새 승인 대상으로 본다.
- 02 — 중단: timeout 뒤 동일 comment를 다시 게시한다.
- 03 — 복구: target·SHA·body hash를 idempotency key로 저장하고 기존 remote result를 재사용한다.
- 04 — 증거: checkpoint\_and\_audit.json과 test 결과를 함께 남긴다.

# Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

같은 plan을 두 번 실행해  
publisher call이 1회인지 test한  
다.

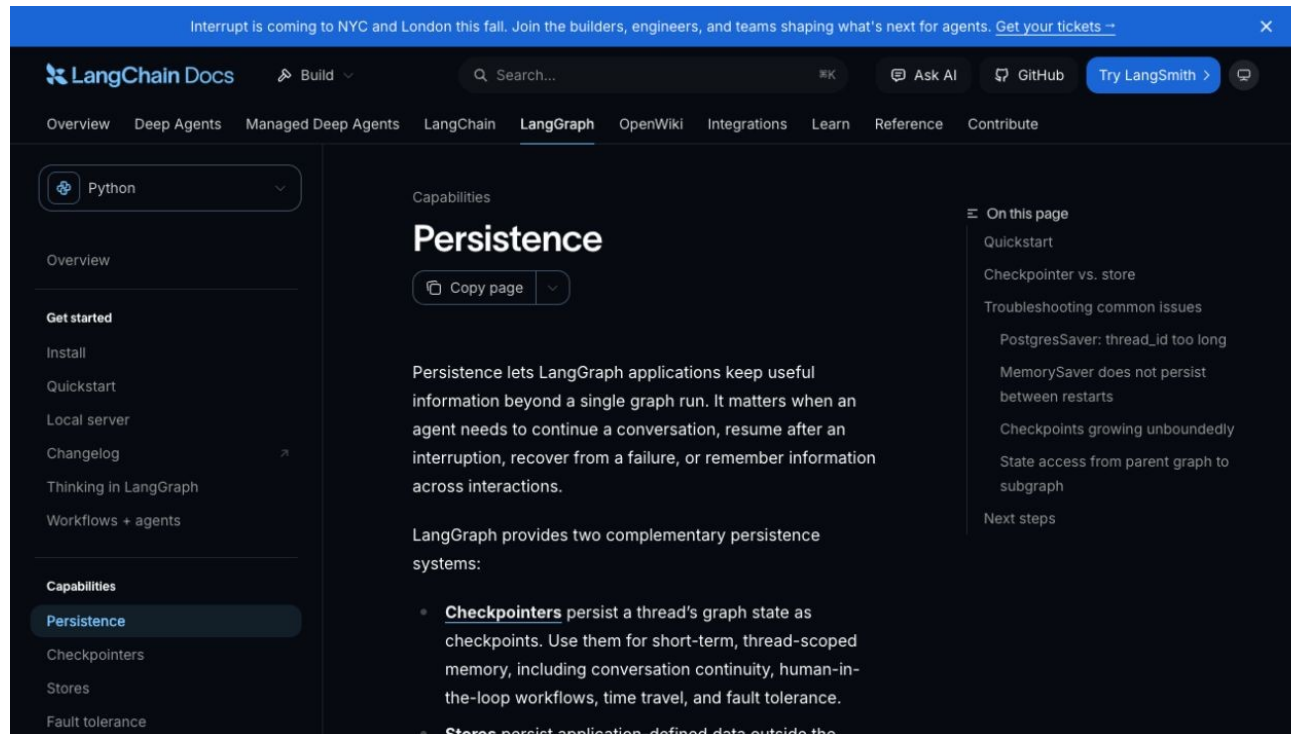
저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge



# Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

같은 plan을 두 번 실행해 publisher call이 1회인지 test한다.

허용 경로

src/langgraph\_lab.py  
tests/test\_course\_services.py

완료 조건

첫 실행은 reused=false, 두 번째는 true다.  
body나 head SHA가 달라지면 새 승인 대상으로 본다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

# 생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01      **Scope**                      —      허용 경로 밖 파일이 바뀌지 않았는가
- 02      **Focused test**                —      첫 실행은 reused=false, 두 번째는 true다.
- 03      **Boundary test**                —      body나 head SHA가 달라지면 새 승인 대상으로 본다.
- 04      **Human merge**                  —      Codex 리뷰와 test 통과는 자동 승인이 아니다



# 강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: `src/course_services/github_service.py`
- 02 — 구현 확인: `src/langgraph_lab.py`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py -k idempotent`
- 04 — 성공 신호: 첫 실행은 `reused=false`, 두 번째는 `true`다.

## 같은 저장소에서 이 명령을 실행합니다

```
$ python -m pytest -q tests/test_course_services.py -k idempotent
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

# 완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED\_FAILURE

ARTIFACT

checkpoint\_and\_audit.json

사람이 확인할 세 줄

1. 첫 실행은 reused=false, 두 번째는 true다.
2. body나 head SHA가 달라지면 새 승인 대상으로 본다.
3. external\_write=false

## 강사는 실패 한 건도 바로 재현하고 복구합니다

재현

timeout 뒤 동일 comment를 다시  
게시한다.

복구

target·SHA·body hash를 idempotency  
key로 저장하고 기존 remote result를  
재사용한다.

확인: body나 head SHA가 달라지면 새 승인 대상으로 본다.

# 이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

materials/day4/day4\_service\_lab.ipynb

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

첫 실행은 reused=false, 두 번째는 true다.  
checkpoint\_and\_audit.json 저장

## STEP 1 · 입력과 설정을 확인합니다

파일 : `src/course_services/github_service.py`  
변수 : 실행 경로·provider·dry-run 여부

---

결과 파일 : `checkpoint_and_audit.json`

## STEP 2 · 정상 경로를 실행합니다

```
python -m pytest -q tests/test_course_services.py -k idempotent
```

성공 신호: 첫 실행은 reused=false, 두 번째는 true다.

---

결과 파일: checkpoint\_and\_audit.json

## STEP 3 · 경계값을 바꿔 실패를 확인합니다

body나 head SHA가 달라지면 새 승인 대상으로 본다.

복구: target·SHA·body hash를 idempotency key로 저장하고 기존 remote result를 재사용한다.

---

결과에 error\_code와 external\_write=false가 보이면 완료



## 정상 case를 focused test로 확인합니다

NORMAL

첫 실행은 reused=false, 두 번째는 true다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

## 가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

body나 head SHA가 달라지면 새 승인 대상으로 본다.

- 01 — raw traceback 대신 stable error\_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

# 재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

## 네트워크가 불안한 CI bot의 중복 comment 방지

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — checkpoint\_and\_audit.json을 운영 검토 자료로 사용

# 구직자는 제품 판단과 검증 증거를 함께 보여줍니다

## 네트워크가 불안한 CI bot의 중복 comment 방지

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — checkpoint\_and\_audit.json과 README 재현 절차를 제출

# 서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

## 품질

---

무엇을 첫 실행은 reused=false, 두 번째는 true다.로 정의하는가

## 안전

---

어디에서 사람 승인과 external\_write=false를 확인하는가

## 복구

---

target-SHA-body hash를 idempotency key로 저장하고 기존 remote result를 재사용한다.

## 관측

---

checkpoint\_and\_audit.json에서 어떤 status:error\_code를 보는가

## 5차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: 재시작 가능한 workflow는 같은 일을 두 번 하지 않습니다이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py -k idempotent`
- 03 — 증거: `checkpoint_and_audit.json`과 정상·실패 test 결과가 남아 있다.

다음 차시: Human Approval은 버튼보다 검토할 정보가 중요합니다

# Human Approval은 버튼보다 검토할 정보가 중요합니다

approve·edit·reject interrupt payload를 설계한다.

---

진행

강의 12분 · 강사 시연 10분 · 소프트웨어  
실습 23분 · 실행 확인 5분

사용 파일

src/langgraph\_lab.py  
src/meeting\_agent\_workflow.py

확인할 결과

review\_decision.json

## 6차시는 앞의 결과를 이어받아 다음 상태를 만듭니다

|       |                                    |
|-------|------------------------------------|
| 입력    | checkpoint_and_audit.json          |
| 이번 차시 | Human Approval은 버튼보다 검토할 정보가 중요합니다 |
| 결과    | review_decision.json               |
| 다음 연결 | Dry-run은 실제 게시 직전의 완성된 결과여야 합니다    |



초안만 보여주고 대상과 근거를 숨기면 사람은 사실상 승인할 수 없다.

approve·edit·reject interrupt payload를 설계한다.

---

이번 차시의 확인 결과: review\_decision.json

# 이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — approve·edit·reject interrupt payload를 설계한다.
- 02 — interrupt를 일반 Exception으로 잡아 자동 진행한다. 왜 위험한지 설명한다.
- 03 — review\_decision.json에서 정상과 실패 상태를 직접 확인한다.

## 처음 보는 용어는 이 네 가지 역할만 구분합니다

| 용어        | 이 수업에서 쓰는 뜻           |
|-----------|-----------------------|
| Interrupt | workflow를 멈추고 입력을 기다림 |
| Approve   | 초안을 그대로 허용            |
| Edit      | 사람이 수정 후 허용           |
| Reject    | 외부 쓰기 없이 종료           |

# 입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

---

src/langgraph\_lab.py

OUTPUT

---

review\_decision.json

PROCESS

---

초안 생성 → target 표시 → finding 근거

VERIFY

---

approve가 READY\_FOR\_EXPORT로 간다.  
reject는 external\_write=false와 REJECTED로 끝난다.

## 실행 흐름은 다섯 단계로 고정합니다

01

초안 생성

02

target 표시

03

finding 근거

04

사람 결정

05

같은 thread resume

마지막 판단은 review\_decision.json에서 사람이 확인합니다.

# 코드보다 먼저 성공·실패 계약을 정합니다

|       |                                                  |
|-------|--------------------------------------------------|
| 정상    | —— approve가 READY_FOR_EXPORT로 간다.                |
| 경계    | —— reject는 external_write=false와 REJECTED로 끝난다.  |
| 외부 쓰기 | —— 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출  |
| 기록    | —— status·error_code·입력 식별자·review_decision.json |

# 어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

```
src/langgraph_lab.py
```

구현 코드

```
src/meeting_agent_workflow.py
```

검증·test

```
web-demo/app.js
```

따라하기 notebook

```
materials/day4/day4_service_lab.ipynb
```

## Human Approval은 버튼보다 검토할 정보가 중요합니다

초안만 보여주고 대상과 근거를 숨기면 사람은 사실상 승인할 수 없다.

interrupt를 workflow event로 두고 같은 thread\_id와 JSON decision으로 resume한다.

이 차시에서  
버릴 오해

interrupt를 일반  
Exception으로 잡아  
자동 진행한다.



## 비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

| 구분        | 역할                    | 이번 차시의 판단 기준     |
|-----------|-----------------------|------------------|
| Interrupt | workflow를 멈추고 입력을 기다림 | 결정론 test로 먼저 확인  |
| Approve   | 초안을 그대로 허용            | adapter 경계를 확인   |
| Edit      | 사람이 수정 후 허용           | 비용·지연·권한을 확인     |
| Reject    | 외부 쓰기 없이 종료           | 사람 판단과 운영 기록을 확인 |

# 가장 먼저 재현할 실패는 이것입니다

실패

interrupt를 일반 Exception으로 잡아  
자동 진행한다.

사용자 영향

초안만 보여주고 대상과 근거를 숨기면  
사람은 사실상 승인할 수 없다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

## 복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: reject는 external\_write=false와 REJECTED로 끝난다.
- 02 — 중단: interrupt를 일반 Exception으로 잡아 자동 진행한다.
- 03 — 복구: interrupt를 workflow event로 두고 같은 thread\_id와 JSON decision으로 resume한다.
- 04 — 증거: review\_decision.json과 test 결과를 함께 남긴다.

# Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

승인 node 전에 외부 쓰기 호출이 존재하지 않는지 diff review한다.

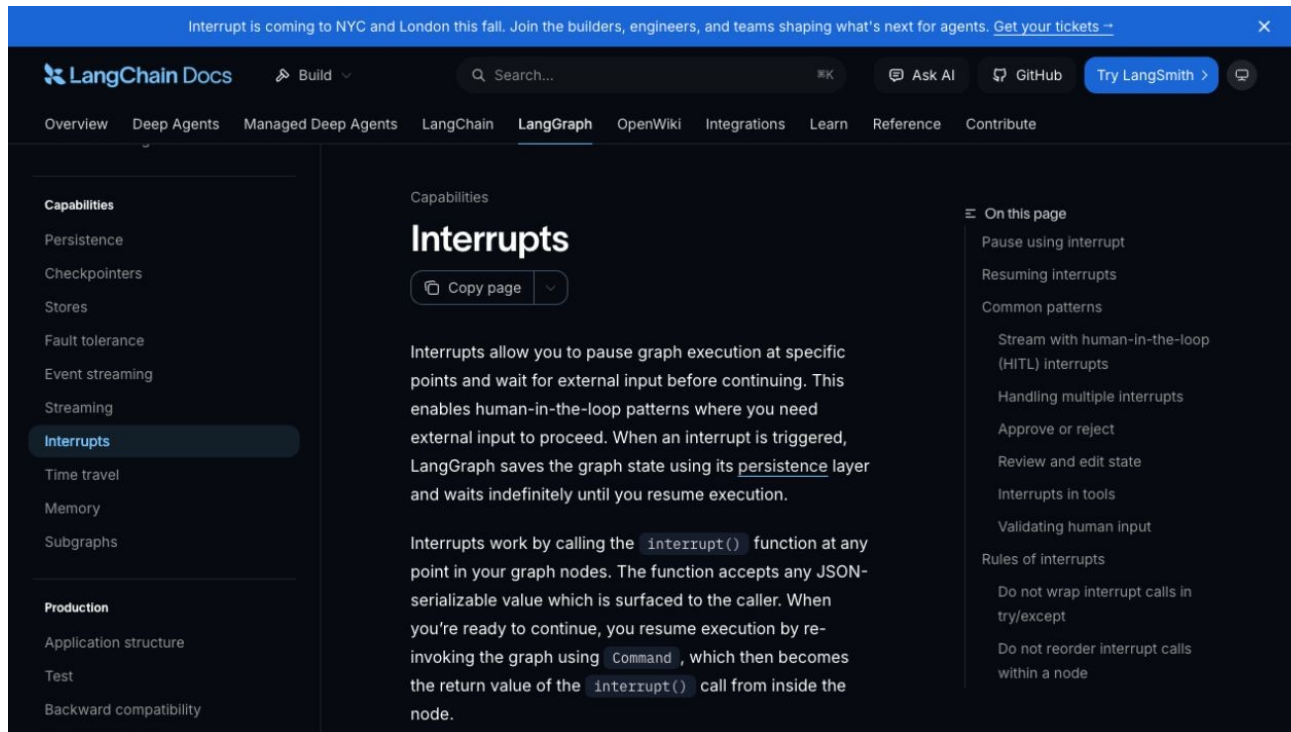
저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge



# Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

승인 node 전에 외부 쓰기 호출이 존재하지 않는지 diff review한다.

허용 경로

src/meeting\_agent\_workflow.py  
web-demo/app.js

완료 조건

approve가 READY\_FOR\_EXPORT로 간다.  
reject는 external\_write=false와 REJECTED로 끝난다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

# 생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- |    |               |    |                                              |
|----|---------------|----|----------------------------------------------|
| 01 | Scope         | —— | 허용 경로 밖 파일이 바뀌지 않았는가                         |
| 02 | Focused test  | —— | approve가 READY_FOR_EXPORT로 간다.               |
| 03 | Boundary test | —— | reject는 external_write=false와 REJECTED로 끝난다. |
| 04 | Human merge   | —— | Codex 리뷰와 test 통과는 자동 승인이 아니다                |

# 강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: `src/langgraph_lab.py`
- 02 — 구현 확인: `src/meeting_agent_workflow.py`
- 03 — 실행: `python -m src.langgraph_lab --decision all`
- 04 — 성공 신호: `approve`가 `READY_FOR_EXPORT`로 간다.

## 같은 저장소에서 이 명령을 실행합니다

```
$ python -m src.langgraph_lab --decision all
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지



# 완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED\_FAILURE

ARTIFACT

review\_decision.json

사람이 확인할 세 줄

1. approve가 READY\_FOR\_EXPORT로 간다.
2. reject는 external\_write=false와 REJECTED로 끝난다.
3. external\_write=false

# 강사는 실패 한 건도 바로 재현하고 복구합니다

## 재현

interrupt를 일반 Exception으로 잡아  
자동 진행한다.

## 복구

interrupt를 workflow event로 두고  
같은 thread\_id와 JSON decision으로  
resume한다.

확인: reject는 external\_write=false와 REJECTED로 끝난다.

# 이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

materials/day4/day4\_service\_lab.ipynb

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

approve가 READY\_FOR\_EXPORT로 간다.  
review\_decision.json 저장

## STEP 1 · 입력과 설정을 확인합니다

파일: src/langgraph\_lab.py  
변수: 실행 경로·provider·dry-run 여부

---

결과 파일: review\_decision.json

## STEP 2 · 정상 경로를 실행합니다

```
python -m src.langgraph_lab --decision all
```

성공 신호: approve가 READY\_FOR\_EXPORT로 간다.

---

결과 파일: review\_decision.json

## STEP 3 · 경계값을 바꿔 실패를 확인합니다

reject는 `external_write=false`와 `REJECTED`로 끝난다.

복구: `interrupt`를 workflow event로 두고 같은 `thread_id`와 `JSON decision`으로 `resume`한다.

---

결과에 `error_code`와 `external_write=false`가 보이면 완료

## 정상 case를 focused test로 확인합니다

NORMAL

approve가 READY\_FOR\_EXPORT로 간다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

## 가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

reject는 `external_write=false`와 REJECTED로 끝난다.

- 01 — raw traceback 대신 stable error\_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가



# 재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

## PR comment·메일·ticket 생성 전에 동일 승인 UI 사용

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — review\_decision.json을 운영 검토 자료로 사용

# 구직자는 제품 판단과 검증 증거를 함께 보여줍니다

## PR comment·메일·ticket 생성 전에 동일 승인 UI 사용

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — review\_decision.json과 README 재현 절차를 제출

# 서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

## 품질

---

무엇을 approve가 READY\_FOR\_EXPORT로 간다.로 정의하는가

## 안전

---

어디에서 사람 승인과 external\_write=false를 확인하는가

## 복구

---

interrupt를 workflow event로 두고 같은 thread\_id와 JSON decision으로 resume한다.

## 관측

---

review\_decision.json에서 어떤 status\_error\_code를 보는가

## 6차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: Human Approval은 버튼보다 검토할 정보가 중요하다는 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m src.langgraph_lab --decision all`
- 03 — 증거: `review_decision.json`과 정상·실패 test 결과가 남아 있다.

다음 차시: Dry-run은 실제 게시 직전의 완성된 결과여야 합니다

# Dry-run은 실제 게시 직전의 완성된 결과여야 합니다

PR comment body-target-idempotency key를 local JSON으로 만든다.

---

| 진행                                           | 사용 파일                                                                     | 확인할 결과                   |
|----------------------------------------------|---------------------------------------------------------------------------|--------------------------|
| 강의 12분 · 강사 시연 10분 · 소프트웨어 실습 23분 · 실행 확인 5분 | src/course_services/github_service.py<br>src/course_services/contracts.py | review_comment_plan.json |

## 7차시는 앞의 결과를 이어받아 다음 상태를 만듭니다

|       |      |                                       |
|-------|------|---------------------------------------|
| 입력    | ———— | review_decision.json                  |
| 이번 차시 | ———— | Dry-run은 실제 게시 직전의 완성된 결과여야 합니다       |
| 결과    | ———— | review_comment_plan.json              |
| 다음 연결 | ———— | 실제 GitHub 쓰기는 본인 sandbox에서 한 건만 실행합니다 |

대충 만든 preview는 실제 API payload와 달라 승인 증거가 되지 못한다.

PR comment body·target·idempotency key를 local JSON  
으로 만든다.

---

이번 차시의 확인 결과: review\_comment\_plan.json

# 이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — PR comment body·target·idempotency key를 local JSON으로 만든다.
- 02 — dry\_run=true인데 publisher가 호출된다. 왜 위험한지 설명한다.
- 03 — review\_comment\_plan.json에서 정상과 실패 상태를 직접 확인한다.



## 처음 보는 용어는 이 네 가지 역할만 구분합니다

| 용어             | 이 수업에서 쓰는 뜻            |
|----------------|------------------------|
| Dry-run        | 외부 쓰기 없이 동일 payload 생성 |
| Payload        | API에 보낼 구조화 데이터        |
| Audit          | 누가 무엇을 승인했는지 기록        |
| External write | 외부 시스템 상태를 바꾸는 호출      |

# 입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

---

src/course\_services/github\_service.py

PROCESS

---

report validate → comment render → target bind

OUTPUT

---

review\_comment\_plan.json

VERIFY

---

payload에 repo·PR·SHA·body·key가 있다.  
dry-run plan은 approval=true여도 publish되지 않는다.

## 실행 흐름은 다섯 단계로 고정합니다

01

report validate

02

comment render

03

target bind

04

hash key

05

READY\_FOR\_APPROVAL

마지막 판단은 review\_comment\_plan.json에서 사람이 확인합니다.

# 코드보다 먼저 성공·실패 계약을 정합니다

|       |                                                      |
|-------|------------------------------------------------------|
| 정상    | —— payload에 repo·PR·SHA·body·key가 있다.                |
| 경계    | —— dry-run plan은 approval=true여도 publish되지 않는다.      |
| 외부 쓰기 | —— 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출      |
| 기록    | —— status·error_code·입력 식별자·review_comment_plan.json |

# 어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

```
src/course_services/github_service.py
```

구현 코드

```
src/course_services/contracts.py
```

검증·test

```
tests/test_course_services.py
```

따라하기 notebook

```
materials/day4/day4_service_lab.ipynb
```

## Dry-run은 실제 게시 직전의 완성된 결과여야 합니다

대충 만든 preview는 실제 API payload와 달라 승인 증거가 되지 못한다.

DRY\_RUN\_CANNOT\_PUBLISH를 contract로 고정하고 fake publisher call 수를 assert한다.

이 차시에서  
버릴 오해

`dry_run=true`인데  
publisher가 호출된다.

## 비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

| 구분             | 역할                     | 이번 차시의 판단 기준     |
|----------------|------------------------|------------------|
| Dry-run        | 외부 쓰기 없이 동일 payload 생성 | 결정론 test로 먼저 확인  |
| Payload        | API에 보낼 구조화 데이터        | adapter 경계를 확인   |
| Audit          | 누가 무엇을 승인했는지 기록        | 비용·지연·권한을 확인     |
| External write | 외부 시스템 상태를 바꾸는 호출      | 사람 판단과 운영 기록을 확인 |

# 가장 먼저 재현할 실패는 이것입니다

실패

`dry_run=true`인데 publisher가 호출된다.

사용자 영향

대충 만든 preview는 실제 API payload와 달라 승인 증거가 되지 못한다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.



## 복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: dry-run plan은 approval=true여도 publish되지 않는다.
- 02 — 중단: dry\_run=true인데 publisher가 호출된다.
- 03 — 복구: DRY\_RUN\_CANNOT\_PUBLISH를 contract로 고정하고 fake publisher call 수를 assert한다.
- 04 — 증거: review\_comment\_plan.json과 test 결과를 함께 남긴다.

# Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

dry-run 경계를 건드리는  
변경에는 boundary test 없이는  
완료하지 않는다.

저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge

The screenshot shows a 'WORKFLOW LAB' interface. At the top, there's a navigation bar with 'Pipeline', 'Review', and 'Result'. The main content area is titled '03 COMMAND RESUME' and has a large heading '같은 thread의 최종 상태를 확인합니다.' Below this, there are five boxes: 'TERMINAL STATUS REJECTED' (with a note about not being sent out and ending in a rejected state), 'RELEASE GATE HOLD' (with a note about schema and release gate checks), 'SIDE EFFECT LOCAL JSON ONLY' (with a note about not performing email, payment, and customer data changes), 'AUDIT EVENTS' (listing 'validate REVIEW\_REQUIRED', 'review reject', and 'rejected'), and 'OUTPUT' (titled '내 결정이 담긴 JSON' with a note about saving demo results as a file, Git commit, or portfolio evidence).

# Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

dry-run 경계를 건드리는 변경에는 boundary test 없이는 완료하지 않는다.

허용 경로

src/course\_services/contracts.py  
tests/test\_course\_services.py

완료 조건

payload에 repo·PR·SHA·body·key가 있다.  
dry-run plan은 approval=true여도 publish되지 않는다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

# 생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01      **Scope**                      —      허용 경로 밖 파일이 바뀌지 않았는가
- 02      **Focused test**              —      payload에 repo·PR·SHA·body·key가 있다.
- 03      **Boundary test**              —      dry-run plan은 approval=true여도 publish되지 않는다.
- 04      **Human merge**              —      Codex 리뷰와 test 통과는 자동 승인이 아니다

# 강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: `src/course_services/github_service.py`
- 02 — 구현 확인: `src/course_services/contracts.py`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py -k dry_run`
- 04 — 성공 신호: payload에 `repo·PR·SHA·body·key`가 있다.

## 같은 저장소에서 이 명령을 실행합니다

```
$ python -m pytest -q tests/test_course_services.py -k dry_run
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

# 완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED\_FAILURE

ARTIFACT

review\_comment\_plan.json

사람이 확인할 세 줄

1. payload에 repo·PR·SHA·body·key가 있다.
2. dry-run plan은 approval=true여도 publish 되지 않는다.
3. external\_write=false

## 강사는 실패 한 건도 바로 재현하고 복구합니다

재현

`dry_run=true`인데 publisher가 호출된다.

복구

`DRY_RUN_CANNOT_PUBLISH`를 contract로 고정하고 fake publisher call 수를 assert한다.

확인: dry-run plan은 `approval=true`여도 publish되지 않는다.



# 이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

materials/day4/day4\_service\_lab.ipynb

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

payload에 repo·PR·SHA·body·key가 있다.  
review\_comment\_plan.json 저장

## STEP 1 · 입력과 설정을 확인합니다

파일 : `src/course_services/github_service.py`  
변수 : 실행 경로·provider·dry-run 여부

---

결과 파일 : `review_comment_plan.json`

## STEP 2 · 정상 경로를 실행합니다

```
python -m pytest -q tests/test_course_services.py -k dry_run
```

성공 신호: payload에 repo·PR·SHA·body·key가 있다.

---

결과 파일: review\_comment\_plan.json

## STEP 3 · 경계값을 바꿔 실패를 확인합니다

dry-run plan은 approval=true여도 publish되지 않는다.

복구: DRY\_RUN\_CANNOT\_PUBLISH를 contract로 고정하고 fake publisher call 수를 assert한다.

---

결과에 error\_code와 external\_write=false가 보이면 완료

## 정상 case를 focused test로 확인합니다

NORMAL

payload에 repo·PR·SHA·body·key가 있다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

## 가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

dry-run plan은 approval=true여도 publish되지 않는다.

- 01 — raw traceback 대신 stable error\_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

# 재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

## 운영자가 Slack/PR 화면에서 실제 게시 내용을 사전 검토

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — review\_comment\_plan.json을 운영 검토 자료로 사용

# 구직자는 제품 판단과 검증 증거를 함께 보여줍니다

## 운영자가 Slack/PR 화면에서 실제 게시 내용을 사전 검토

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — review\_comment\_plan.json과 README 재현 절차를 제출



# 서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

## 품질

---

무엇을 payload에 repo·PR·SHA·body·key가 있다.로 정의하는가

## 안전

---

어디에서 사람 승인과 external\_write=false를 확인하는가

## 복구

---

DRY\_RUN\_CANNOT\_PUBLISH를 contract로 고정하고 fake publisher call 수를 assert한다.

## 관측

---

review\_comment\_plan.json에서 어떤 status·error\_code를 보는가

## 7차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: Dry-run은 실제 게시 직전의 완성된 결과여야 합니다이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py -k dry_run`
- 03 — 증거: `review_comment_plan.json`과 정상·실패 test 결과가 남아 있다.

다음 차시: 실제 GitHub 쓰기는 본인 sandbox에서 한 건만 실행합니다

16:20-17:10 (쉬는 시간 17:10-17:40)

# 실제 GitHub 쓰기는 본인 sandbox에서 한 건만 실행합니다

선택적으로 승인된 comment 한 건을 게시하고 audit ID를 남긴다.

---

## 진행

강의 12분 · 강사 시연 10분 · 소프트웨어  
실습 23분 · 실행 확인 5분

## 사용 파일

materials/day4/day4\_service\_lab.ipynb  
src/course\_services/github\_service.py

## 확인할 결과

day4\_audit\_record.json

## 8차시는 앞의 결과를 이어받아 다음 상태를 만듭니다

|       |     |                                       |
|-------|-----|---------------------------------------|
| 입력    | ——— | review_comment_plan.json              |
| 이번 차시 | ——— | 실제 GitHub 쓰기는 본인 sandbox에서 한 건만 실행합니다 |
| 결과    | ——— | day4_audit_record.json                |
| 다음 연결 | ——— | Q&A·실행 복구·release 판단                  |

교육 실습이 타인 저장소나 업무 repo를 오염시키면 안 된다.

선택적으로 승인된 comment 한 건을 게시하고 audit ID를 남긴다.

---

이번 차시의 확인 결과: day4\_audit\_record.json

# 이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — 선택적으로 승인된 comment 한 건을 게시하고 audit ID를 남긴다.
- 02 — 교육 코드에 live publisher와 token scope를 기본 포함한다. 왜 위험한지 설명한다.
- 03 — day4\_audit\_record.json에서 정상과 실패 상태를 직접 확인한다.

## 처음 보는 용어는 이 네 가지 역할만 구분합니다

| 용어           | 이 수업에서 쓰는 뜻                 |
|--------------|-----------------------------|
| Sandbox repo | 실습 전용 본인 저장소                |
| Publisher    | 외부 API write adapter        |
| Remote ID    | 게시 결과 식별자                   |
| Rollback     | comment 삭제 또는 commit revert |

# 입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

---

materials/day4/day4\_service\_lab.ipynb

OUTPUT

---

day4\_audit\_record.json

PROCESS

---

sandbox 확인 → dry-run 검토 → 사람 승인

VERIFY

---

fake publisher가 한 번 호출되고 remote ID가 audit에 남는다.  
publisher 미설정·승인 없음·dry-run은 모두 BLOCKED  
다.



## 실행 흐름은 다섯 단계로 고정합니다

01

sandbox 확인

02

dry-run 검토

03

사람 승인

04

1회 POST

05

response ID 기록

마지막 판단은 day4\_audit\_record.json에서 사람이 확인합니다.

# 코드보다 먼저 성공·실패 계약을 정합니다

|       |                                                 |
|-------|-------------------------------------------------|
| 정상    | fake publisher가 한 번 호출되고 remote ID가 audit에 남는다. |
| 경계    | publisher 미설정·승인 없음·dry-run은 모두 BLOCKED다.       |
| 외부 쓰기 | 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출    |
| 기록    | status·error_code·입력 식별자·day4_audit_record.json |

# 어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

```
materials/day4/day4_service_lab.ipynb
```

구현 코드

```
src/course_services/github_service.py
```

검증·test

```
.env.sample
```

따라하기 notebook

```
AGENTS.md
```

## 실제 GitHub 쓰기는 본인 sandbox에서 한 건만 실행합니다

교육 실습이 타인 저장소나 업무 repo를 오염시키면 안 된다.

repo에는 fake publisher만 두고 live adapter는 강사 승인 후 별도 branch에서 연결한다.

이 차시에서  
버릴 오해

교육 코드에 live  
publisher와 token scope  
를 기본 포함한다.

## 비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

| 구분           | 역할                          | 이번 차시의 판단 기준     |
|--------------|-----------------------------|------------------|
| Sandbox repo | 실습 전용 본인 저장소                | 결정론 test로 먼저 확인  |
| Publisher    | 외부 API write adapter        | adapter 경계를 확인   |
| Remote ID    | 게시 결과 식별자                   | 비용·지연·권한을 확인     |
| Rollback     | comment 삭제 또는 commit revert | 사람 판단과 운영 기록을 확인 |

# 가장 먼저 재현할 실패는 이것입니다

실패

교육 코드에 live publisher와 token scope를 기본 포함한다.

사용자 영향

교육 실습이 타인 저장소나 업무 repo를 오염시키면 안 된다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

## 복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: publisher 미설정·승인 없음·dry-run은 모두 BLOCKED다.
- 02 — 중단: 교육 코드에 live publisher와 token scope를 기본 포함한다.
- 03 — 복구: repo에는 fake publisher만 두고 live adapter는 강사 승인 후 별도 branch에서 연결한다.
- 04 — 증거: day4\_audit\_record.json과 test 결과를 함께 남긴다.

# Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

PR 생성·리뷰·test 수정은 돕되  
merge와 외부 게시 결정은  
사람에게 남긴다.

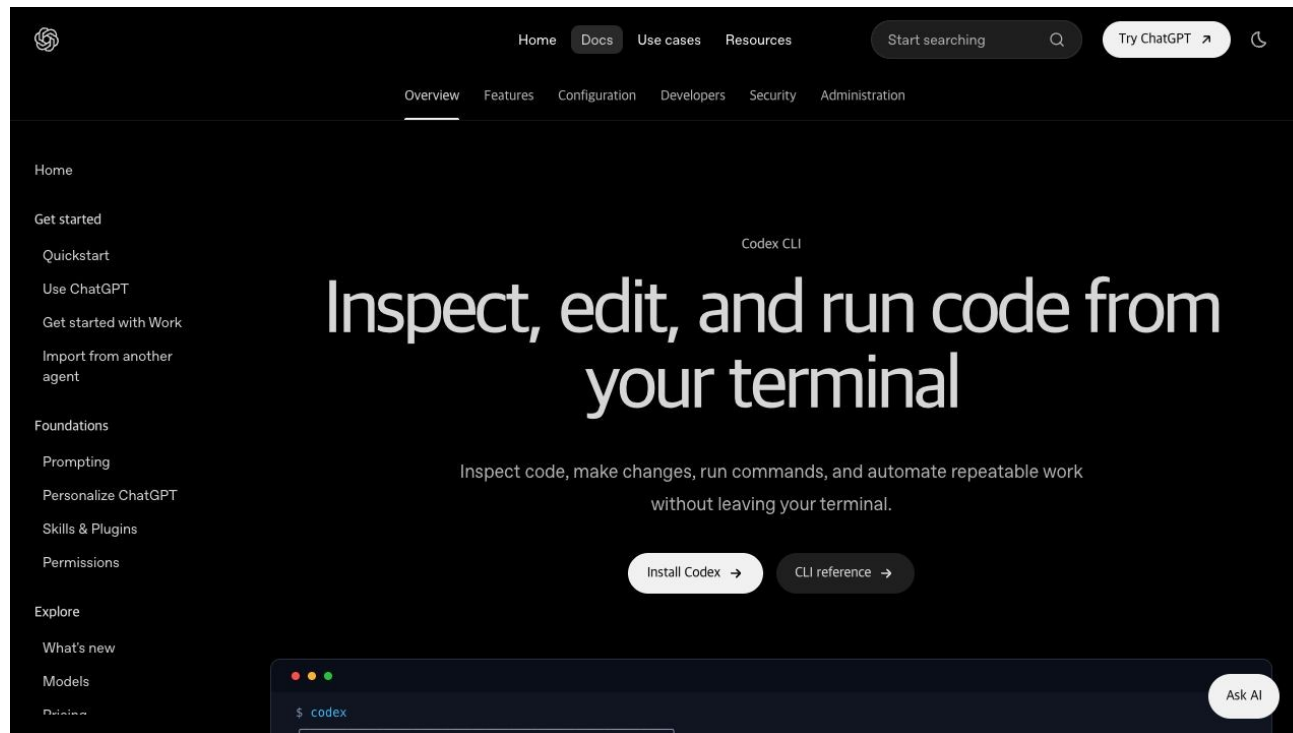
저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge





# Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

PR 생성·리뷰·test 수정은 돕되 merge와 외부 게시 결정은 사람에게 남긴다.

허용 경로

src/course\_services/github\_service.py  
.env.sample

완료 조건

fake publisher가 한 번 호출되고 remote ID가 audit에 남는다.  
publisher 미설정·승인 없음·dry-run은 모두 BLOCKED다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

# 생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01      **Scope**                      —      허용 경로 밖 파일이 바뀌지 않았는가
- 02      **Focused test**              —      fake publisher가 한 번 호출되고 remote ID가 audit에 남는다.
- 03      **Boundary test**              —      publisher 미설정·승인 없음·dry-run은 모두 BLOCKED다.
- 04      **Human merge**              —      Codex 리뷰와 test 통과는 자동 승인이 아니다

# 강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: `materials/day4/day4_service_lab.ipynb`
- 02 — 구현 확인: `src/course_services/github_service.py`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py`
- 04 — 성공 신호: fake publisher가 한 번 호출되고 remote ID가 audit에 남는다.

## 같은 저장소에서 이 명령을 실행합니다

```
$ python -m pytest -q tests/test_course_services.py
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

# 완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED\_FAILURE

ARTIFACT

day4\_audit\_record.json

사람이 확인할 세 줄

1. fake publisher가 한 번 호출되고 remote ID가 audit에 남는다.
2. publisher 미설정·승인 없음·dry-run은 모두 BLOCKED다.
3. external\_write=false

## 강사는 실패 한 건도 바로 재현하고 복구합니다

재현

교육 코드에 live publisher와 token scope를 기본 포함한다.

복구

repo에는 fake publisher만 두고 live adapter는 강사 승인 후 별도 branch에서 연결한다.

확인: publisher 미설정·승인 없음·dry-run은 모두 BLOCKED다.

# 이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

AGENTS.md

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

fake publisher가 한 번 호출되고 remote ID가 audit에 남는다.  
day4\_audit\_record.json 저장

## STEP 1 · 입력과 설정을 확인합니다

파일: materials/day4/day4\_service\_lab.ipynb  
변수: 실행 경로·provider·dry-run 여부

---

결과 파일: day4\_audit\_record.json



## STEP 2 · 정상 경로를 실행합니다

```
python -m pytest -q tests/test_course_services.py
```

성공 신호: fake publisher가 한 번 호출되고 remote ID가 audit에 남는다.

---

결과 파일: day4\_audit\_record.json

## STEP 3 · 경계값을 바꿔 실패를 확인합니다

publisher 미설정·승인 없음·dry-run은 모두 BLOCKED다.

복구: repo에는 fake publisher만 두고 live adapter는 강사 승인 후 별도 branch  
에서 연결한다.

---

결과에 error\_code와 external\_write=false가 보이면 완료

## 정상 case를 focused test로 확인합니다

NORMAL

fake publisher가 한 번 호출되고 remote ID가 audit에 남는다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

## 가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

**publisher 미설정·승인 없음·dry-run은 모두 BLOCKED다.**

- 01 — raw traceback 대신 stable error\_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

# 재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

현업은 GitHub App과 protected branch, 구직자는 sandbox demo recording

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — day4\_audit\_record.json을 운영 검토 자료로 사용

# 구직자는 제품 판단과 검증 증거를 함께 보여줍니다

현업은 GitHub App과 protected branch, 구직자는 sandbox demo recording

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — day4\_audit\_record.json과 README 재현 절차를 제출

# 서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

## 품질

---

무엇을 fake publisher가 한 번 호출되고 remote ID가 audit에 남는다.로 정의하는가

## 안전

---

어디에서 사람 승인과 external\_write=false를 확인하는가

## 복구

---

repo에는 fake publisher만 두고 live adapter는 강사 승인 후 별도 branch에서 연결한다.

## 관측

---

day4\_audit\_record.json에서 어떤 status\_error\_code를 보는가

## 8차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: 실제 GitHub 쓰기는 본인 sandbox에서 한 건만 실행합니다! 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py`
- 03 — 증거: `day4_audit_record.json`과 정상·실패 test 결과가 남아 있다.

17:10-17:40 쉬는 시간 · 17:40-18:00 Q&A·실행 복구